```
from google.colab import drive
drive.mount('/content/drive')

```

```
Mounted at /content/drive
```

```
# 2. Add the directory containing 'models.py' to Python's path
import sys
sys.path.append('/content/drive/MyDrive/') # Assuming 'models.py' is directly in MyDrive

# 3. Import the module
import models
from models import *
```

```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_quant_8x8"
model = VGG16_quant_8x8()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243, 0.262])


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2)


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2)


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()
```

```
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))



def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out the status. e.g., i%5 => every 5 batch,
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
```

```
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120 epochs"""
    adjust_list = [150, 225]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
=> Building model...
VGG_quant(
  (features): Sequential(
    (0): QuantConv2d(
      3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): QuantConv2d(
      64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU(inplace=True)
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (7): QuantConv2d(
      64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (8): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (9): ReLU(inplace=True)
    (10): QuantConv2d(
      128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (11): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (12): ReLU(inplace=True)
    (13): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (14): QuantConv2d(
      128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (15): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```
    (16): ReLU(inplace=True)
    (17): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (18): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (19): ReLU(inplace=True)
    (20): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (21): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (22): ReLU(inplace=True)
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (24): QuantConv2d(
      256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (25): BatchNorm2d(8, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (26): ReLU(inplace=True)
    (27): QuantConv2d(
      8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
```

```
model.features[27] ## Modified layer
```

```
QuantConv2d(
  8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

```
# import os
# os.getcwd()
```

```
PATH = "/content/drive/MyDrive/result/VGG16_quant_8x8/model_best.pth.tar"
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 3159/10000 (32%)
```

```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

######### Save inputs from selected layer ##########
save_output = SaveOutput()
i = 0
count = 0
for layer in model.modules():
    i = i+1
    if isinstance(layer, QuantConv2d):
        count += 1
        print(i,"-th layer prehooked \n", layer, count) #'count' here is just for me to check the index of save_output.
        layer.register_forward_pre_hook(save_output)
####################################################
model.cuda()
dataiter = iter(testloader)
```

```
images, labels = next(dataiter)
images = images.cuda()
out = model(images)
```

```
3 -th layer prehooked
QuantConv2d(
  3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 1
7 -th layer prehooked
QuantConv2d(
  64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 2
12 -th layer prehooked
QuantConv2d(
  64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 3
16 -th layer prehooked
QuantConv2d(
  128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 4
21 -th layer prehooked
QuantConv2d(
  128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 5
25 -th layer prehooked
QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 6
29 -th layer prehooked
QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 7
34 -th layer prehooked
QuantConv2d(
  256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 8
38 -th layer prehooked
QuantConv2d(
  8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 9
41 -th layer prehooked
QuantConv2d(
  8, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 10
46 -th layer prehooked
QuantConv2d(
  256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 11
50 -th layer prehooked
QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
```

```
weight_q = model.features[27].weight_q
w_alpha = model.features[27].weight_quant.wgt_alpha
w_bit = 4

weight_int = weight_q / (w_alpha / (2**(w_bit-1)-1))
#print(weight_int)
```

```
act = save_output.outputs[8][0]
act_alpha  = model.features[27].act_alpha
act_bit = 4
act_quant_fn = act_quantization(act_bit)

act_q = act_quant_fn(act, act_alpha)

act_int = act_q / (act_alpha / (2**act_bit-1))
#print(act_int )
```

```
act_int.shape
```

```
torch.Size([128, 8, 4, 4])
```

```
save_output.outputs[9][0].shape
```

```
torch.Size([128, 8, 4, 4])
```

```
conv_int = torch.nn.Conv2d(in_channels = 8, out_channels=8, kernel_size = 3, padding=1)
conv_int.weight = torch.nn.parameter.Parameter(weight_int)
relu = torch.nn.ReLU()
conv_int.bias = model.features[0].bias
output_int = conv_int(act_int)
output_recovered = output_int * (act_alpha / (2**act_bit-1)) * (w_alpha / (2**(w_bit-1)-1))
output_recovered_relu = relu(output_recovered)
#print(output_recovered)
```

```
difference = abs(save_output.outputs[9][0] - output_recovered_relu )
print(difference.mean())  ## It should be small, e.g.,2.3 in my trainned model
```

```
tensor(0.2977, device='cuda:0', grad_fn=<MeanBackward0>)
```

```
# print(output_int)
```

```

# act_int.size = torch.Size([128, 64, 32, 32])  <- batch_size, input_ch, ni, nj
a_int = act_int[2,:,:,:]  # pick only one input out of batch
# a_int.size() = [64, 32, 32]
print(a_int.shape)
# conv_int.weight.size() = torch.Size([64, 64, 3, 3])  <- output_ch, input_ch, ki, kj
w_int = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))  # merge ki, kj index to kij
# w_int.weight.size() = torch.Size([64, 64, 9])
print(w_int.shape)
```

```
torch.Size([8, 4, 4])
torch.Size([8, 8, 9])
```

```

padding = 1
stride = 1
array_size = 8 # row and column number

```

```

nig = range(a_int.size(1))  ## ni group
njg = range(a_int.size(2))  ## nj group
print(nig, njg)
```

```
range(0, 4) range(0, 4)
```

```

icg = range(int(w_int.size(1)))  ## input channel
ocg = range(int(w_int.size(0)))  ## output channel
print(icg, ocg)
```

```
range(0, 8) range(0, 8)
```

```

ic_tileg = range(int(len(icg)/array_size))
oc_tileg = range(int(len(ocg)/array_size))
print(ic_tileg, oc_tileg)
```

```
range(0, 1) range(0, 1)
```

```

kijg = range(w_int.size(2))
ki_dim = int(math.sqrt(w_int.size(2)))  ## Kernel's 1 dim size
print(kijg, ki_dim)
```

```
range(0, 9) 3
```

```

######## Padding before Convolution #######
a_pad = torch.zeros(len(icg), len(nig)+padding*2, len(nig)+padding*2).cuda()
print(a_pad.shape)
```

```
torch.Size([8, 6, 6])
```

```


a_pad[ :, padding:padding+len(nig), padding:padding+len(njg)] = a_int.cuda()
```

```
a_pad = torch.reshape(a_pad, (a_pad.size(0), -1))
print(a_pad.shape)
```

```
torch.Size([8, 36])
```

```


a_tile = torch.zeros(len(ic_tileg), array_size,    a_pad.size(1)).cuda()
w_tile = torch.zeros(len(oc_tileg)*len(ic_tileg), array_size, array_size, len(kijg)).cuda()
print(a_tile.shape)
print(w_tile.shape)
```

```
torch.Size([1, 8, 36])
torch.Size([1, 8, 8, 9])
```

```

for ic_tile in ic_tileg:
    a_tile[ic_tile,:,:] = a_pad[ic_tile*array_size:(ic_tile+1)*array_size,:]

for ic_tile in ic_tileg:
    for oc_tile in oc_tileg:
        w_tile[oc_tile*len(oc_tileg) + ic_tile,:,:,:] = w_int[oc_tile*array_size:(oc_tile+1)*array_size, ic_tile*array_
```

```

##########################################

p_nijg = range(a_pad.size(1)) ## psum nij group

psum = torch.zeros(len(ic_tileg), len(oc_tileg), array_size, len(p_nijg), len(kijg)).cuda()

for kij in kijg:
    for ic_tile in ic_tileg:       # Tiling into array_sizeXarray_size array
        for oc_tile in oc_tileg:   # Tiling into array_sizeXarray_size array
            for nij in p_nijg:       # time domain, sequentially given input
                    m = nn.Linear(array_size, array_size, bias=False)
                    #m.weight = torch.nn.Parameter(w_int[oc_tile*array_size:(oc_tile+1)*array_size, ic_tile*array_size:
                    m.weight = torch.nn.Parameter(w_tile[len(oc_tileg)*oc_tile+ic_tile,:,:,kij])
                    psum[ic_tile, oc_tile, :, nij, kij] = m(a_tile[ic_tile,:,nij]).cuda()

```

```
print(oc_tileg)
```

```
range(0, 1)
```

```
import math

a_pad_ni_dim = int(math.sqrt(a_pad.size(1))) # 32
print(a_pad_ni_dim)
```

```
6
```

```

o_ni_dim = int((a_pad_ni_dim - (ki_dim- 1) - 1)/stride + 1)
o_nijg = range(o_ni_dim**2)
print(o_ni_dim, o_nijg)
```

```
4 range(0, 16)
```

```

out = torch.zeros(len(ocg), len(o_nijg)).cuda()


### SFP accumulation ###
for o_nij in o_nijg:
    for kij in kijg:
        for ic_tile in ic_tileg:
            for oc_tile in oc_tileg:
                out[oc_tile*array_size:(oc_tile+1)*array_size, o_nij] = out[oc_tile*array_size:(oc_tile+1)*array_size,
                psum[ic_tile, oc_tile, :, int(o_nij/o_ni_dim)*a_pad_ni_dim + o_nij%o_ni_dim + int(kij/ki_dim)*a_pad_ni_
                #print(int(o_nij/o_ni_dim)*a_pad_ni_dim + o_nij%o_ni_dim + int(kij/ki_dim)*a_pad_ni_dim + kij%ki_dim)
                ## 4th index = (int(o_nij/30)*32 + o_nij%30) + (int(kij/3)*32 + kij%3)
```

```
out_2D = torch.reshape(out, (out.size(0), o_ni_dim, -1))
difference = (out_2D - output_int[2,:,:,:])
```

```
print(difference.sum())
```

```
tensor(0.0003, device='cuda:0', grad_fn=<SumBackward0>)
```

```
print(out.shape)
# print(relu(out))
```

```
torch.Size([8, 16])
```

```
print(output_int.shape)
print(out.shape)
print(a_tile.shape)

```

```
torch.Size([128, 8, 4, 4])
torch.Size([8, 16])
torch.Size([1, 8, 36])
```

```
####### Store activation ####################
tile_id = 0
nij = 0 # just a random number
X = a_tile[tile_id,:,:]  # [tile_num, array row num, time_steps]

bit_precision = 8 #changed from 4 to 8 to accomodate new sizing
file = open('activation_tile0.txt', 'w') #write to file
file.write('#time0row7[msb-lsb],time0row6[msb-lst],....,time0row0[msb-lst]#\n')
file.write('#time1row7[msb-lsb],time1row6[msb-lst],....,time1row0[msb-lst]#\n')
file.write('#.............#\n')
print(torch.max(X))
for i in range(X.size(1)):  # time step
    for j in range(X.size(0)): # row #
        X_bin = '{0:08b}'.format(round(X[7-j,i].item())) #X_bin = '{0:04b}'.format(round(X[7-j,i].item()))
        for k in range(bit_precision):
            #print(X_bin[k])
            file.write(X_bin[k])
        #file.write(' ')  # for visibility with blank between words, you can use
    file.write('\n')
file.close() #close file

```

```
tensor(13., device='cuda:0', grad_fn=<MaxBackward1>)
```

```
print(w_tile.shape)
```

```
torch.Size([1, 8, 8, 9])
```

```
####### store weight(kij) ###############
tile_id = 0
kij = 0
bit_precision = 4


for kij in range (9):
    file_name = f'weight_itile0_otile0_kij{kij}.txt'
    W = w_tile[tile_id,:,:,kij] # w_tile[tile_num, array col num, array row num, kij]

    file = open(file_name, 'w')  #write to file
    file.write('#col0row7[msb-lsb],col0row6[msb-lst],....,col0row0[msb-lst]#\n')
    file.write('#col1row7[msb-lsb],col1row6[msb-lst],....,col1row0[msb-lst]#\n')
    file.write('#.............#\n')

    for i in range(W.size(1)):  # time step
        for j in range(W.size(0)): # row #
            if (W[i, 7-j].item() >= 0):
                W_bin = '{0:04b}'.format(round(W[i, 7-j].item()))
            else:
                W_bin = '{0:04b}'.format(round(W[i, 7-j].item())+16)
            for m in range(2):
              for k in range(bit_precision):
                  file.write(W_bin[k])
            #file.write(' ')  # for visibility with blank between words, you can use
        file.write('\n')
    file.close() #close file
```

```
# ### Store psum ###
# ic_tile_id = 0
# oc_tile_id = 0


# kij = 0
```

```
# nij = 0

# # psum[len(ic_tileg), len(oc_tileg), array_size, len(p_nijg), len(kijg)]


# bit_precision = 16
# file = open('psum.txt', 'w') #write to file
# file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
# file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
# file.write('#.................#\n')

# for kij in range(9):
#     psum_tile = psum[ic_tile_id,oc_tile_id,:,nij:nij+36,kij]
#     for i in range(psum_tile.size(1)):  # time step
#         for j in range(psum_tile.size(0)): # row #
#             if (psum_tile[7-j, i].item() >= 0):
#                 psum_tile_bin = '{0:016b}'.format(round(psum_tile[7-j,i].item()))
#             else:
#                 psum_tile_bin = '{0:016b}'.format(round(psum_tile[7-j,i].item())+65536)
#             #print(W_bin,' ', W[7-j,i].item(), j, i, '\n')
#             for k in range(bit_precision):
#                 file.write(psum_tile_bin[k])
#             #file.write(' ')  # for visibility with blank between words, you can use
#         file.write('\n')
# file.close() #close file
```

```
sfpout: 0000000000001110111111111110000010000000000000000000000001000111111111101101101000000000000000011111111111110
answer: 0000000000001110000000000000000000000000000000000000000001000110000000000000000000000000000000000000000000000
sfpout: 1111111110101100111111111011101000000000000000000111111110101111111111111101110100000000000000001111111100111
answer: 0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
```

```
# psum_tile.shape
# print(psum_tile)
# print(out)
```

```
print(out.shape)
print(output_int.shape)
out_int = output_int[2,:,:,:]
out_int = out_int.reshape(out_int.size(0),-1)
print(out_int.shape) #out_ch, o_nij
print(relu(out_int))
```

```
torch.Size([8, 16])
torch.Size([128, 8, 4, 4])
torch.Size([8, 16])
tensor([[0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         1.4000e+01, 0.0000e+00, 0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 2.1000e+01, 0.0000e+00, 0.0000e+00,
         2.8000e+01, 6.3000e+01, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 7.0000e+00,
         9.5367e-07, 3.5000e+01, 2.1000e+01, 3.5000e+01],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 0.0000e+00, 1.4000e+01, 0.0000e+00, 0.0000e+00, 0.0000e+00,
         7.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00],
        [0.0000e+00, 0.0000e+00, 0.0000e+00, 5.6000e+01, 0.0000e+00, 0.0000e+00,
         0.0000e+00, 7.0000e+00, 0.0000e+00, 0.0000e+00, 0.0000e+00, 2.8000e+01,
         0.0000e+00, 4.2000e+01, 3.5000e+01, 1.4000e+01]], device='cuda:0',
       grad_fn=<ReluBackward0>)
```

```
bit_precision = 16
file = open('out.txt', 'w') #write to file
file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
file.write('#.................#\n')

out_relu = relu(out_int)
#print(out_relu.shape)

```

```
for i in range(16): #o_nij
    #print(out_relu[:,i])
    for j in range(8): #out_channel
        #Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item())+65536)
        if (out_relu[7-j, i].item() >= 0):
            Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item()))
        else:
            Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item())+65536)
        for k in range(bit_precision):
            file.write(Out_bin[k])
    file.write('\n')
file.close() #close file
```

```
##### The following code is just for verification #####
##### Please Ignore #####
file = open('out.txt', 'r')
result = []

# Obtain the value in the last column
for line in file:
    line = line.strip()
    if not line or line.startswith("#"):
        continue
    parts = line.split()
    last_col = parts[-1]
    result.append(last_col)

print('The last column in out.txt is\n')
print(result)

def twos_complement_to_int(num_str, bits=16):
    num_str = num_str.zfill(bits)
    num = int(num_str, 2)
    if num >= 2**(bits-1):
        num -= 2**bits
    return num

print('\nConvert binary number into decimal number\n')
result_int = [twos_complement_to_int(x) for x in result]
print(result_int)
```

```
The last column in out.txt is

['0000000000000000000000000000000000000000000011100000000000000000000000000000111000000000000011100000000000000000000000000
Convert binary number into decimal number

[33849922953150326695198720, 42312403686512081244192768, 42312403686512081244192768, 42312403686512081244192768, 4231240368
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.